

Simulación con ARMSim

Enlace de descarga:

<https://webhome.cs.uvic.ca/~nigelh/ARMSim-V2.1/Windows/index.html>

Instalación:

Descargar Installer.msi y ejecutarlo mediante doble click.

Aceptar los términos de la licencia GNU.

Advanced

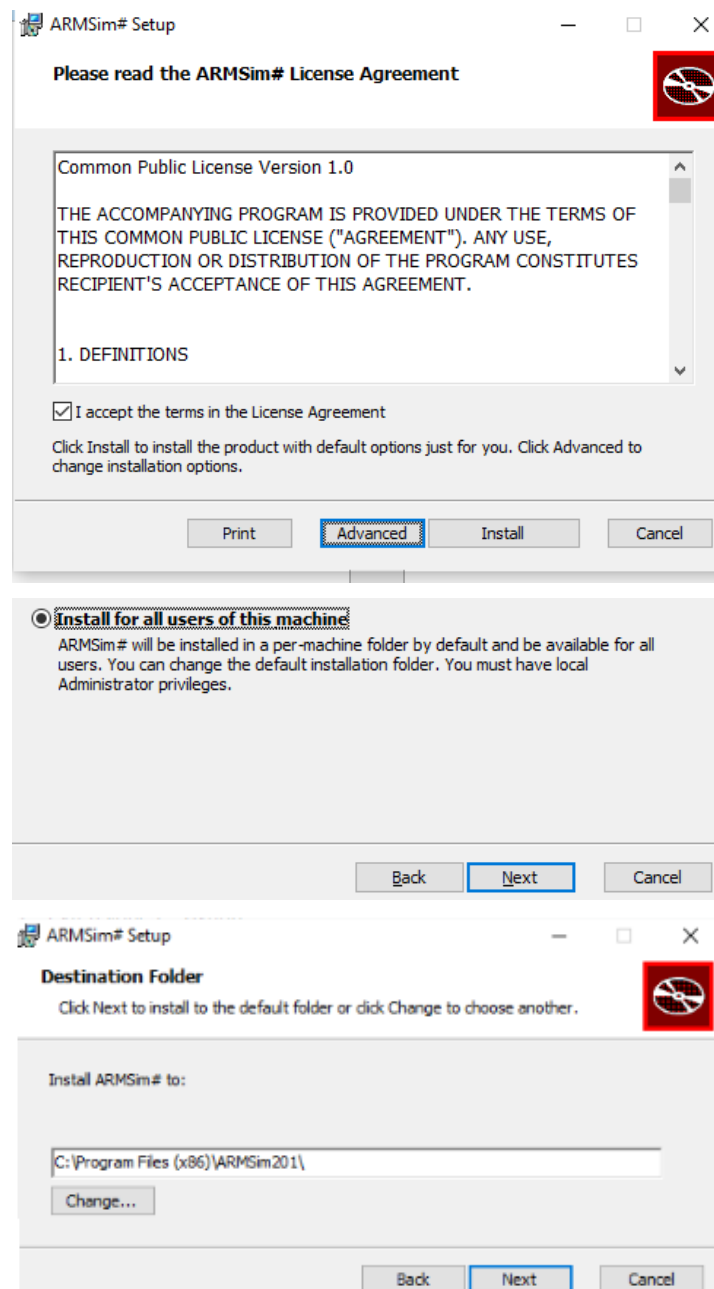
Instalar para todos los usuarios

Next

Elegir la carpeta de destino donde se va a instalar.

Next.

Install





Primer programa

Sumar dos números.

```

1  @ En este programa realizamos la suma de dos números.
2  @ Los operandos pasarán de variables a registros.
3  @ El resultado pasará de un registro a una variable.
4  @ Veremos los registros, la memoria y, mediante watchview, las variables.
5  @ 1. cargar el programa
6  @ 2. view memory
7
8      .equ SWI_Exit, 0x11    @ constante
9
10     .data                  @ variables
11
12     numero1:    .word 2
13     numero2:    .word 5
14     resultado:  .word 0
15
16     .text            @ sección de código
17
18     .global main      @ avisa al linker que main es global
19
20 main:
21     ldr r0, =numero1    @ r0 = dirección de numero1 para direccionamiento directo
22     ldr r1, [r0]         @ r1 = 2 carga r1 con modo de direccionamiento directo
23     ldr r0, =numero2    @ r0 = dirección de numero2 para direccionamiento directo
24     ldr r2, [r0]         @ r2 = 5 carga r2 con modo de direccionamiento directo
25     add r3, r1, r2       @ suma r1 + r2 y lo guarda en r3
26     ldr r0, =resultado   @ r0 = dirección de resultado
27     str r3, [r0]         @ resultado = r3 guarda r3 en la variable resultado
28
29     swi SWI_Exit        @ salir del programa
30     .end

```

@ En este programa realizamos la suma de dos números.
 @ Los operandos pasarán de variables a registros.
 @ El resultado pasará de un registro a una variable.
 @ Veremos los registros, la memoria y, mediante WatchView, las variables.
 @ 1. cargar el programa
 @ 2. view memory

```

.equ SWI_Exit, 0x11    @ constante

.data                  @ variables
numero1:    .word 2
numero2:    .word 5
resultado:  .word 0

.text            @ sección de código
.global main      @ avisa al linker que main es global

```

```

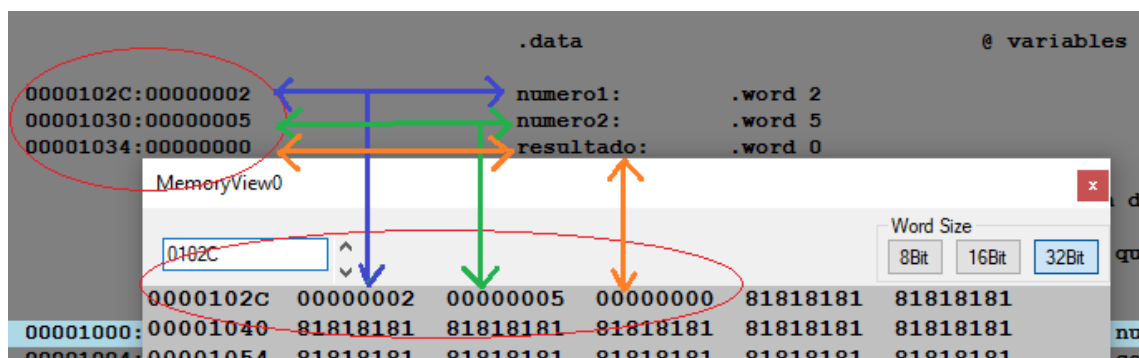
main:
    ldr r0, =numero1    @ r0 = dirección de numero1 para direccionamiento directo

```

ldr r1, [r0]	@ r1 = 2 carga r1 con modo de direccionamiento directo
ldr r0, =numero2	@ r0 = dirección de numero2 para direccionamiento directo
ldr r2, [r0]	@ r2 = 5 carga r2 con modo de direccionamiento directo
add r3, r1, r2	@ suma r1 + r2 y lo guarda en r3
ldr r0, =resultado	@ r0 = dirección de resultado
str r3, [r0]	@ resultado = r3 guarda r3 en la variable resultado
swi SWI_Exit	@ salir del programa
.end	

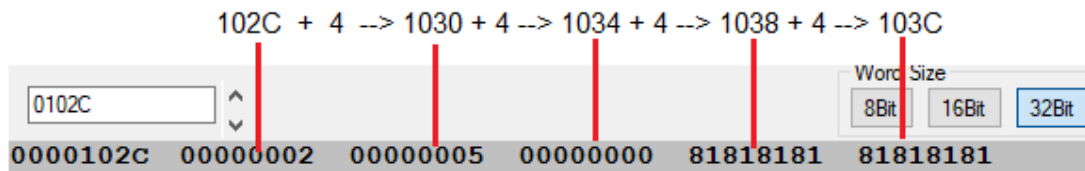
Procedimiento de prueba

1. Escribir el programa en un editor de texto o en un IDE y guardarlo con extensión .s en una carpeta de programas ARM. El simulador ARMSim no cuenta con un editor.
2. Abrir ARMSim
3. Cargar el programa File → Load → 01_suma.s
4. Observar en la ventana de code view que la dirección de la variable munero1 es 0102C
5. En View → Memory abrir MemoryView()
6. Poner la dirección 0102C en MemoryView
7. Observar los datos de las variables almacenadas en memoria:

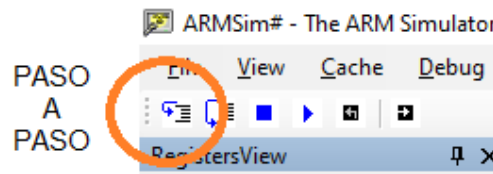


Observar en la MemoryView() que el 2 está en la dirección 102C y que la siguiente dirección, la que contiene al 5, es la 1030 porque los números de direcciones de instrucciones consecutivas van de 4 en 4, dado que la memoria principal es de 8 bits y las variables declaradas son 32 bits (.word).

De igual manera, el resultado se va a almacenar en la dirección 1034 y la siguiente línea comienza en la dirección 1040:



8. Observar que el registro R0 contiene 0 y ejecutar la primera instrucción mediante el botón paso a paso:

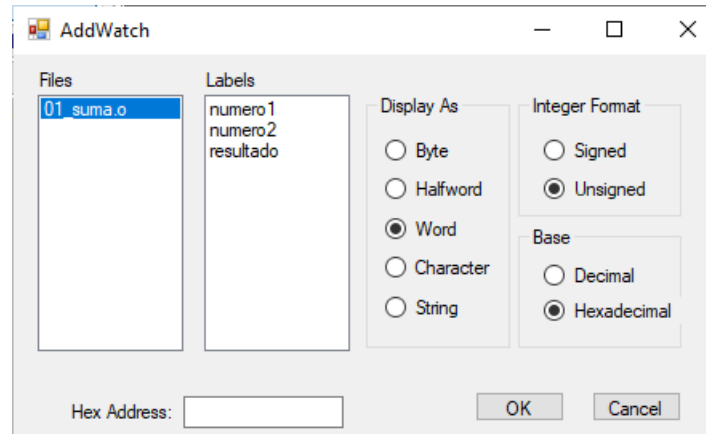


ldr r0, =numero1 hace que, ahora, R0 contenga la dirección 102C de la variable numero1

9. Agregar variables a la ventana WatchView():

Watch → Add Watch

Seleccionar la etiqueta de la variable que queremos ver y pulsar OK.



10. Ir ejecutando las instrucciones siguientes paso a paso. Observar en cada paso el nuevo contenido de los registros, relacionar el contenido de los registros con el contenido de la memoria en la dirección dada por las variables, observar el resultado en la memoria y en la ventana WatchView()

Limitaciones

En el help del menú puede verse la versión de ARM:

```
ARMSim# Version 2.0.1 (4)
University of Victoria
Produced by:
Dr. Nigel Horspool
Dale Lyons
Dr. Micaela Serra
Bill Bird
Department of Computer Science.

Copyright 2006--2017 University of Victoria.
Simulating ARMv5 instruction architecture with Vector
Floating Point support and a Data/Instruction Cache
simulation.
```

Hay instrucciones que son de versiones posteriores y no pueden simularse. Por ejemplo, la división entera UDIV:

```
Error: selected processor does not support ARM mode `udiv r3,r1,r2'
```

Enlaces útiles

ARM Developers: <https://developer.arm.com/>

Documentación: <https://developer.arm.com/documentation>

ISA:

<https://developer.arm.com/documentation/ddi0602/2023-09/Base-Instructions?lang=en>